

Take-Home Assignment — Support Ticket & SLA (Service Level Agreement) Tracker

Objective

Build a simple **Support Ticket & SLA (Service Level Agreement) Tracker** where users can raise tickets, agents (agent refers to support staff not AI agent) can work on them, and every ticket has an SLA deadline calculated using **business hours only**.

You may design the UI in your own way.

The most important business rule is:

**SLA time is measured in business hours, not wall-clock hours.
Nights, weekends, and holidays do not count against an SLA.**

The assignment is intended to evaluate application architecture, GraphQL API design, database modeling, business-logic implementation, testing, and code quality

Requirements

1. Runtime & Language

- **Runtime — Bun / Node js**
 - **Language — TypeScript**
 - TypeScript must run in **strict mode**
 - **any** is not allowed
 - Use proper TypeScript types throughout the application
 - Configure appropriate type-checking and linting
-

2. API — GraphQL Yoga

Use **GraphQL Yoga** for the backend API.

The API must be **schema-first**:

- Define the GraphQL schema using `.graphql` files
- Implement resolvers separately in TypeScript
- Do not use a code-first GraphQL schema approach

The API should support at least the following core types:

None

```
type Ticket {  
  id: ID!  
  title: String!  
  description: String!  
  priority: Priority!  
  status: TicketStatus!  
  reporter: User!  
  assignee: User  
  createdAt: String!  
  firstResponseAt: String  
  resolvedAt: String  
  sla: SLAInfo!  
}
```

```
enum Priority {  
  LOW  
  MEDIUM  
  HIGH  
  URGENT
```

```
}
```

```
enum TicketStatus {
```

```
    OPEN
```

```
    IN_PROGRESS
```

```
    RESOLVED
```

```
    CLOSED
```

```
}
```

Queries

Implement:

```
None
```

```
tickets(
```

```
    status: TicketStatus
```

```
    priority: Priority
```

```
    assigneeId: ID
```

```
    slaState: SLAState
```

```
    take: Int
```

```
    cursor: String
```

```
): TicketConnection!
```

```
ticket(id: ID!): Ticket
```

```
dashboard: TicketDashboard!
```

```
users(role: UserRole): [User!]!
```

```
holidays: [Holiday!]!
```

The exact schema may differ if there is a good architectural reason, but the API must support:

- Paginated ticket listing
- Filtering by status
- Filtering by priority
- Filtering by assignee
- Filtering by SLA state
- Fetching an individual ticket
- Dashboard/summary information
- Users/agents
- Configured holidays

3. Mutations

Implement at least:

None

```
createTicket(  
  title: String!  
  description: String!  
  priority: Priority!  
): Ticket!
```

```
assignTicket(  
  ticketId: ID!  
  assigneeId: ID!  
): Ticket!  
  
changeTicketStatus(  
  ticketId: ID!  
  status: TicketStatus!  
): Ticket!  
  
addComment(  
  ticketId: ID!  
  content: String!  
): Comment!  
  
resolveTicket(  
  ticketId: ID!  
): Ticket!
```

Authentication-related mutations:

```
None  
register(  
  name: String!
```

```
    email: String!

    password: String!

    role: UserRole!
  ): AuthPayload!

login(
  email: String!

  password: String!
): AuthPayload!
```

You may restrict role creation appropriately if you prefer not to allow arbitrary users to register as agents.

4. Database — PostgreSQL + Prisma

Use:

- **PostgreSQL**
- **Prisma ORM**
- **Docker Compose** for PostgreSQL

The database schema must be managed using Prisma migrations.

Requirements:

- Use `prisma migrate dev`
- Commit migration files to the repository
- Do not manually modify the database schema using SQL
- Add sensible indexes for commonly filtered fields
- Use proper Prisma relations and constraints

The core models should include:

None

User

Ticket

Comment

Holiday

A ticket must contain at least:

- title
- description
- priority
- status
- reporter
- optional assignee
- creation timestamp
- first response timestamp
- resolution timestamp

Comments must belong to a ticket and record their author.

5. Authentication & Authorization

Implement user authentication.

The system must support at least two roles:

None

REPORTER

AGENT

Authentication requirements:

- User registration
- User login
- Passwords must never be stored in plain text
- Use a secure password hashing mechanism such as Argon2 or bcrypt
- Authenticated users can create tickets
- Agents can perform agent-specific actions

- Authorization must be enforced server-side

For example:

- A reporter can create tickets and comment
- An agent can assign tickets
- An agent can change ticket status
- An agent can resolve tickets

The exact permission model is up to you as long as it is enforced by the backend.

6. Ticket Validation

All validation must happen server-side.

Reject invalid requests with proper GraphQL errors rather than unhandled 500 responses.

At minimum:

- Empty ticket title
- Empty ticket description
- Invalid priority
- Empty comment
- Non-existent ticket
- Non-existent assignee
- Invalid status transition
- Unauthorized operation

Use meaningful, machine-readable error codes where appropriate.

For example:

None

VALIDATION_ERROR

TICKET_NOT_FOUND

USER_NOT_FOUND

UNAUTHORIZED

FORBIDDEN

INVALID_STATUS_TRANSITION

7. Ticket Status Transitions

Ticket status changes must follow sensible rules.

At minimum, prevent invalid transitions such as:

None

CLOSED → IN_PROGRESS

unless the ticket has first been explicitly reopened.

You may define your own transition rules, but they must be enforced server-side.

For example:

None

OPEN

↓

IN_PROGRESS

↓

RESOLVED

↓

CLOSED

The candidate should document the chosen transition rules in the README.

A rejected transition should return a clear GraphQL error such as:

None

INVALID_STATUS_TRANSITION

Ticket cannot transition from CLOSED to IN_PROGRESS.

8. SLA Engine

This is the most important part of the assignment.

SLA calculations must be implemented **server-side**.

The SLA engine should be isolated into a dedicated, testable module/service and must not be scattered across GraphQL resolvers.

SLA Policies

Use the following default policies:

Priority	First Response	Resolution
URGENT	1 business hour	4 business hours
HIGH	4 business hours	24 business hours
MEDIUM	8 business hours	48 business hours
LOW	24 business hours	72 business hours

You may make these configurable, but these should be the default values.

Business Hours

Business hours are:

None

Monday – Friday

09:00 – 18:00

Therefore:

None

9 business hours per working day

Time outside business hours does not count toward an SLA.

Weekends do not count.

Configured public holidays do not count.

Business hours should use **one configured timezone**.

For example:

None

BUSINESS_TIMEZONE=Asia/Kolkata

The exact configured timezone may be chosen by the candidate.

SLA Due Times

For every ticket, the backend must calculate:

- First response due time
- Resolution due time
- First response SLA state

- Resolution SLA state
- Remaining business time

For example:

None

Ticket:

Priority: HIGH

Created:

Friday 17:00

First response target:

4 business hours

Calculation:

None

Friday 17:00 → 18:00

1 business hour

Saturday

0

Sunday

0

Monday 09:00 → 12:00

3 business hours

Therefore:

None

First response due:

Monday 12:00

SLA State

Each SLA clock should have one of:

None

ON_TRACK

AT_RISK

BREACHED

Rules:

None

ON_TRACK

0% – 75% of SLA budget consumed

AT_RISK

>75% of SLA budget consumed

BREACHED

SLA deadline has passed

The exact boundary behavior at 75% should be documented.

SLA Clock Freezing

Once an SLA event actually happens, its clock must stop.

For example:

None

Ticket created:

Monday 09:00

First response due:

Monday 13:00

Agent responds:

Monday 11:00

The first-response SLA is now completed.

It must **never later become BREACHED**, even if the ticket remains open for several days.

Store the actual event timestamp, for example:

None

firstResponseAt

Similarly, once a ticket is resolved:

None

`resolvedAt`

should freeze the resolution SLA.

First Response

A ticket has a comment thread.

The **first comment made by someone other than the reporter** counts as the first response.

For example:

None

`Reporter → Comment`

`Reporter → Comment`

`Reporter → Comment`

`Agent → Comment ← first response`

The timestamp of the first agent/non-reporter comment should be recorded as:

None

`firstResponseAt`

Subsequent comments must not modify this timestamp.

Remaining Business Time

The API should return the remaining business time for each active SLA.

For example:

None

```
type SLAInfo {  
  
    firstResponseDueAt: String!  
  
    resolutionDueAt: String!  
  
    firstResponseState: SLAState!  
  
    resolutionState: SLAState!  
  
    firstResponseRemainingMinutes: Int!  
  
    resolutionRemainingMinutes: Int!  
  
}
```

The exact representation is up to the candidate.

The frontend may use this value to display a countdown.

The frontend must **not** calculate whether an SLA is breached.

9. Holiday Calendar

Implement a configurable holiday calendar.

A holiday should contain at least:

None

`id`

`date`

`name`

For example:

None

2026-08-15

Independence Day

The SLA engine must exclude holidays from business-hour calculations.

The holiday calendar must actually affect SLA calculations.

Example:

None

Friday 17:00

↓

Monday is a holiday

↓

Tuesday 09:00

Monday must contribute **zero** business hours.

10. SLA Edge Cases

The implementation should correctly handle cases such as:

Ticket created before business hours

None

Monday 07:00

SLA counting should begin at:

None

Monday 09:00

Ticket created after business hours

None

Monday 20:00

SLA counting should begin at:

None

Tuesday 09:00

Friday evening

None

Friday 17:59

Only one minute should count before the weekend.

Weekend

Tickets created during:

None

Saturday

Sunday

should begin counting from the next business period.

Holiday

If Monday is a configured holiday, the next business period should be Tuesday.

These edge cases should be covered by tests.

11. Timezones

- Store timestamps in **UTC**
- API timestamps should use an unambiguous format such as ISO 8601

- Business hours must use the configured business timezone
- Frontend should display timestamps in the user's local timezone
- SLA calculations must correctly handle the configured timezone

The business-hours timezone should be configurable through an environment variable.

For example:

```
None
BUSINESS_TIMEZONE=Asia/Kolkata
```

12. Pagination & Filtering

Ticket listing must use **cursor-based pagination**.

For example:

```
None
tickets(
  status: TicketStatus
  priority: Priority
  assigneeId: ID
  slaState: SLAState
  take: Int
  cursor: String
): TicketConnection!
```

The response should include enough information to retrieve the next page.

For example:

None

```
type TicketConnection {  
  nodes: [Ticket!]!  
  pageInfo: PageInfo!  
}
```

```
type PageInfo {  
  hasNextPage: Boolean!  
  endCursor: String  
}
```

Tickets should be filterable by:

- Status
- Priority
- Assignee
- SLA state

13. Dashboard

Provide a small dashboard/summary API.

For example:

None

```
type TicketDashboard {  
  openTickets: Int!  
  inProgressTickets: Int!  
  atRiskTickets: Int!
```

```
breachedTickets: Int!  
}
```

The frontend should display these statistics.

14. Frontend

Use **React + TypeScript**.

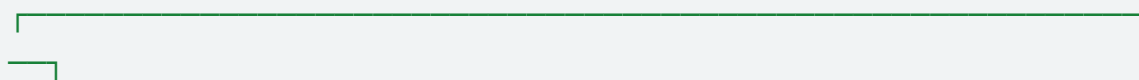
You may use Next.js or another React-based setup if you prefer.

The frontend must:

- Display a ticket list
- Display priority
- Display status
- Display assignee
- Display SLA state
- Display remaining SLA time
- Filter tickets
- Sort tickets
- Create tickets
- View ticket details
- Display comment thread
- Add comments
- Assign tickets
- Change ticket status
- Resolve tickets
- Display dashboard statistics
- Clearly display validation and authorization errors

Example ticket list:

None



Support Tickets				
#1024	Payment failed	URGENT	IN_PROGRESS	
BREACHED				
#1023	Login issue	HIGH	OPEN	 32m
#1022	UI issue	LOW	RESOLVED	 Met

The UI does not need to be visually elaborate.

The primary focus is correctness and usability.

15. Backend-Driven SLA State

The frontend must not determine SLA state using its own business-hour calculations.

The API is the source of truth.

For example, the API should return:

```
JSON
{
  "firstResponseState": "AT_RISK",
  "firstResponseRemainingMinutes": 32
}
```

The frontend can display:

None

⚠ At Risk – 32 minutes remaining

but it must not independently determine:

None

75% consumed → AT_RISK

16. Tests

Automated tests are required.

Unit tests

At minimum, test the SLA/business-hours engine with cases including:

- Normal weekday calculation
- Ticket created before business hours
- Ticket created after business hours
- Weekend
- Friday evening
- Public holiday
- Weekend + holiday combination
- SLA crossing multiple business days
- First-response SLA
- Resolution SLA
- SLA becoming AT_RISK
- SLA becoming BREACHED
- Completed SLA remaining completed

Also test important resolver/business logic such as:

- Ticket creation
- Invalid ticket validation
- Status transitions
- Assignment
- First-response recording
- Comment creation

- Authorization

Integration test

Include at least **one integration test against a real PostgreSQL database running through Docker**.

Do not mock PostgreSQL for this test.

The integration test should exercise the actual Prisma/PostgreSQL persistence layer.

A good candidate for the integration test is:

None

Create ticket

→ Add reporter comment

→ Add agent comment

→ Verify firstResponseAt

→ Verify persisted SLA information

17. Error Handling

Expected business errors must return proper GraphQL errors.

Examples:

None

VALIDATION_ERROR

TICKET_NOT_FOUND

USER_NOT_FOUND

UNAUTHORIZED

FORBIDDEN

INVALID_STATUS_TRANSITION

INVALID_PRIORITY

INVALID_COMMENT

SLA-related errors should also be meaningful if applicable.

Do not allow normal validation failures to become unhandled 500 errors.

18. Project Structure

The exact architecture is up to you, but business logic should be clearly separated from GraphQL resolvers.

For example:

None

src/

 graphql/

 schema/

 resolvers/

 services/

 ticket/

 sla/

 auth/

 repositories/

```
validation/
```

```
auth/
```

```
db/
```

```
server.ts
```

```
prisma/
```

```
  schema.prisma
```

```
  migrations/
```

```
  seed.ts
```

```
tests/
```

```
  unit/
```

```
  integration/
```

The **SLA/business-hours** calculation should be isolated into a dedicated **service/module**.

Do not implement the calculation directly inside GraphQL resolvers.

19. Environment Configuration

Sensitive configuration must not be committed.

Use environment variables such as:

None

`DATABASE_URL`

`JWT_SECRET`

`BUSINESS_TIMEZONE`

Provide:

None

`.env.example`

Do not commit real credentials or secrets.

20. Seed Data

Provide a seed script with enough data to demonstrate the application.

For example:

None

`Users`

`|— reporter@example.com`

`|— agent@example.com`

`Tickets`

`|— URGENT ticket`

`|— HIGH ticket`

└─ MEDIUM ticket

└─ LOW ticket

Holidays

└─ At least one sample holiday

Document the seed credentials in the README if appropriate.

21. README

The repository must contain a meaningful README including:

- Project overview
- Tech stack
- Architecture overview
- Database schema overview
- SLA calculation approach
- Status transition rules
- Authentication approach
- Environment variables
- Setup instructions
- Database migration instructions
- Seed instructions
- How to run the backend
- How to run the frontend
- How to run tests
- Example GraphQL queries/mutations

The project should support a setup flow as close as possible to:

Shell

```
docker compose up -d && bun install && bun run gendb && bun run dev
```

If frontend and backend require separate processes, document the exact commands.

Also include a short section:

How I'd extend this

Briefly describe what you would improve with more time.

Examples:

- SLA pause while waiting for customer
 - Escalation rules
 - Notifications
 - Per-team business calendars
 - Audit logs
 - Agent performance metrics
 - Recurring holidays
 - More sophisticated SLA policies
-

22. Git / GitHub

Submit the assignment through your own Git repository.

Requirements:

- Use **incremental commits**
- Commit messages should describe actual changes
- Do not submit the project as one large commit
- Open a Pull Request against your own `main`
- PR description should include:
 - Implementation summary
 - Architecture decisions
 - SLA calculation approach
 - Tradeoffs
 - Known limitations
 - What would be improved with more time

Avoid meaningless commit messages such as:

None

update

changes

final

done

Prefer messages such as:

None

feat: add ticket and user models

feat: implement GraphQL ticket queries

feat: add business hours SLA calculator

feat: implement ticket status transitions

test: cover holiday and weekend SLA calculations

feat: add ticket dashboard

docs: add setup and architecture documentation

23. Bonus

Optional features include:

- Dockerized application
- Database-level constraints where applicable
- SLA pause while **WAITING_ON_CUSTOMER**
- Audit trail for status/assignee changes
- SLA escalation notifications
- Email notifications
- Per-team business-hour calendars
- Multiple timezones
- Agent performance statistics

- Live-updating countdown
 - End-to-end tests
 - CI pipeline
 - Rate limiting
 - Observability/logging
-

Submission

Submit:

1. Git repository URL
2. Pull Request URL
3. README with setup and architecture documentation
4. 5–10 minute walkthrough, either:
 - Written walkthrough, or
 - Screen recording

The walkthrough should explain:

- Overall architecture
 - GraphQL schema
 - Database schema
 - SLA calculation approach
 - Business-hours handling
 - Timezone handling
 - Status transition design
 - Testing strategy
 - Important tradeoffs
-

Estimated Time

4–6 hours of focused work

Please submit within **3–5 days**.

The focus of this assignment is:

- Problem solving
- TypeScript proficiency
- GraphQL API design

- Database modeling
- Business logic
- SLA/time calculations
- Validation and error handling
- Testing
- Frontend/backend integration
- Code quality